

Graph Traversal

1. explain for bfs graph traversal in c++: cout<<"Enter node and Edge: "; cin>>node>>edge; for(int i=0;i<edge;i++) { cin>>u>>v; adj[u].push_back(v); adj[v].push_back(u); } bfs(0); cout<<"BFS Traversal: "<<endl; for(int i=0;i<res.size();i++) { cout<<res[i]<<" "; }. 2
2. why loop until edge value? for(int i = 0; i < edge; i++) { cin >> u >> v; adj[u].push_back(v); adj[v].push_back(u); }. 6



1. explain for bfs graph traversal in c++:

```
cout<<"Enter node and Edge: ";
cin>>node>>edge;
for(int i=0;i<edge;i++)
{
    cin>>u>>v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
bfs(0);
cout<<"BFS Traversal: "<<endl;
for(int i=0;i<res.size();i++)
{
    cout<<res[i]<<" ";
}
```

Let's go through this BFS (Breadth-First Search) traversal code **line by line**, so you understand what each part does 🙋

Code Breakdown

cpp

```
cout << "Enter node and Edge: ";
cin >> node >> edge;
```

- **Purpose:**

You are taking input for:

- **node:** number of nodes (vertices) in the graph.
- **edge:** number of edges (connections between nodes).

cpp

```
for(int i = 0; i < edge; i++)
{
    cin >> u >> v;
    adj[u].push_back(v);
```

```
adj[v].push_back(u);
}
```

- **Purpose:**

This loop reads all the **edges** of the graph.

- **u** and **v** represent two connected nodes.
- **adj** is an **adjacency list**, usually declared like:

```
cpp
vector<int> adj[100]; // example for max 100 nodes
```

- **adj[u].push_back(v)** means:
"Add **v** to the list of nodes connected to **u**."
- **adj[v].push_back(u)** means:
"Add **u** to the list of nodes connected to **v**."
(This line makes the graph **undirected** — both directions are recorded.)

```
cpp
bfs(0);
```

- **Purpose:**

You are calling the **BFS function**, starting from **node 0**.

- The BFS function typically:
 - Uses a **queue**.
 - Keeps a **visited array** to avoid re-visiting nodes.
 - Traverses the graph **level by level** (like waves).

Example BFS function:

```
cpp
void bfs(int start) {
    queue<int> q;
    vector<bool> visited(node, false);

    q.push(start);
    visited[start] = true;

    while(!q.empty()) {
        int current = q.front();
        q.pop();
        res.push_back(current);
    }
}
```

```

        for(int neighbor : adj[current]) {
            if(!visited[neighbor]) {
                q.push(neighbor);
                visited[neighbor] = true;
            }
        }
    }
}

```

cpp

```

cout << "BFS Traversal: " << endl;
for(int i = 0; i < res.size(); i++)
{
    cout << res[i] << " ";
}

```

- **Purpose:**

Prints the order of nodes visited by the BFS traversal.

- **res** is a vector that stores the sequence of visited nodes.

Example Run

Input:

yaml

```

Enter node and Edge: 5 4
0 1
0 2
1 3
2 4

```

Process:

- Adjacency list:

rust

```

0 -> 1, 2
1 -> 0, 3
2 -> 0, 4
3 -> 1
4 -> 2

```

BFS starting from 0:

- Visit order: 0 → 1 → 2 → 3 → 4

Output:

```
yaml
```

```
BFS Traversal:
0 1 2 3 4
```

Would you like me to show the **full working BFS code** (including all declarations and `main()`)?



2. why loop until edge value?

```
for(int i = 0; i < edge; i++)
{
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

Excellent question 🙌

Let's break it down carefully — the key idea here is how we **input** a graph.

What does an “edge” mean?

In a graph:

- A **node (or vertex)** is a point.
- An **edge** is a connection (or line) **between two nodes**.

Example:

If you have 4 nodes labeled **0**, **1**, **2**, **3**, and 3 edges connecting them like this:

```
0 - 1
1 - 2
2 - 3
```

Then:

- **node** = 4
- **edge** = 3

So why loop until edge?

Each **edge** connects **two nodes** (u and v).

So to **build the adjacency list**, you need to read every connection exactly once.

That's why the loop runs **edge times** — once per edge:

cpp

```
for(int i = 0; i < edge; i++)
{
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

Step-by-step example

Input:

yaml

```
Enter node and Edge: 4 3
0 1
1 2
2 3
```

Loop explanation:

i	Input (u, v)	Action	Adjacency list after this step
0	0 1	connect 0-1	adj[0] = [1], adj[1] = [0]
1	1 2	connect 1-2	adj[1] = [0, 2], adj[2] = [1]
2	2 3	connect 2-3	adj[2] = [1, 3], adj[3] = [2]

After the loop finishes, **all edges are stored** in the adjacency list.



Why not loop until node?

Because:

- Nodes describe **how many vertices exist**.
- But the **connections** are given by the edges.

Even if there are 10 nodes, maybe only 5 of them are connected by 4 edges.
So we must read **exactly edge lines** to describe all connections — not **node** times.

Summary

Concept	Meaning
node	Total number of vertices

Concept	Meaning
edge	Total number of connections
Loop up to edge	To read all connections (u-v pairs)
adj[u].push_back(v)	Connects u $\rightarrow$ v
adj[v].push_back(u)	Connects v $\rightarrow$ u (for undirected graph)

Would you like me to show how this adjacency list looks in memory for a small example (like 5 nodes and 4 edges)?